Real-time Image & Sensors based Driver Drowsiness Detection using Deep Learning

Project Team

Saad Rahman 20P-0027
Aimal Khan 20P-0028

Session 2020-2024

Supervised by

Dr. Hafeez Anwar



Department of Computer Science

National University of Computer and Emerging Sciences
Peshawar, Pakistan

Fall, 2024

Student's Declaration

We declare that this project titled "*Real-time Image & Sensors based Driver Drowsiness Detection using Deep Learning*", submitted as requirement for the award of degree of Bachelors in Computer Science, does not contain any material previously submitted for a degree in any university; and that to the best of our knowledge, it does not contain any materials previously published or written by another person except where due reference is made in the text.

We understand that the management of Department of Computer Science, National University of Computer and Emerging Sciences, has a zero tolerance policy towards plagiarism. Therefore, We, as authors of the above-mentioned thesis, solemnly declare that no portion of our thesis has been plagiarized and any material used in the thesis from other sources is properly referenced.

We further understand that if we are found guilty of any form of plagiarism in the thesis work even after graduation, the University reserves the right to revoke our BS degree.

Saad Rahman Signature: _____

Aimal Khan Signature: _____

_____

Verified by Plagiarism Cell Officer
Dated:

Certificate of Approval



The Department of Computer Science, National University of Computer and Emerging Sciences, accepts this thesis titled *Real-time Image & Sensors based Driver Drowsiness Detection using Deep Learning*, submitted by Saad Rahman (20P-0027), and Aimal Khan (20P-0028), in its current form, and it is satisfying the dissertation requirements for the award of Bachelors Degree in Computer Science.

Supervisor

Dr. Hafeez Anwar Signature: _____

Mr. Haroon Zafar

FYP Coordinator
National University of Computer and Emerging Sciences, Peshawar

---

Mr. Fazle Basit

HoD of Department of Computer Science
National University of Computer and Emerging Sciences

Acknowledgements

We would like to express our deepest gratitude to all those who guided, supported, and encouraged us throughout the journey of completing our Final Year Project, titled "Real-time Image-based Driver Drowsiness Detection using Deep Learning."

First and foremost, our sincerest thanks go to our respected supervisor, Dr. Hafeez Anwar, whose invaluable guidance, unwavering encouragement, and insightful feedback have been instrumental in the successful completion of this project. Dr. Hafeez Anwar's expertise, constructive criticism, and mentorship have greatly enhanced our understanding of the subject matter and significantly influenced the direction and outcome of our work.

We would also like to extend our gratitude to all faculty members and peers who provided thoughtful suggestions and support during various stages of this project. Their feedback and encouragement have been a source of inspiration.

Lastly, we are profoundly grateful to our families for their endless support, patience, and encouragement throughout this journey. Their faith in us has been a constant source of motivation, enabling us to persevere and achieve our objectives.

This project would not have been possible without the collective efforts and contributions of all the above-mentioned individuals. We are deeply thankful for their assistance and support.

Saad Rahman

Aimal Khan

Abstract

Driver drowsiness is a critical factor contributing to road accidents, particularly among long-route drivers, necessitating reliable real-time detection mechanisms. This project presents a comprehensive solution utilizing deep learning to detect drowsiness by integrating image and sensor data in a mobile application. The system employs MobileNet for analyzing webcam-captured images to classify the driver's eye state and CNN-LSTM architecture for processing sensor data, including gyroscope and accelerometer readings, to assess driving behavior.

The methodology involves collecting and preprocessing real-time datasets, training and evaluating the models, and implementing a responsive alert mechanism to warn drivers. The application, developed using Flutter, provides a user-friendly interface to facilitate cross-platform accessibility and real-time functionality. Key technologies such as TensorFlow Lite, OpenCV, and mobile sensor integration enable the seamless execution of deep learning models within the application.

Experimental results highlight the system's accuracy in detecting drowsiness and aggressive driving behaviors, offering timely alerts to prevent accidents. This project demonstrates the feasibility of leveraging advanced deep learning techniques and mobile technology for enhancing road safety. It contributes to reducing the risks associated with drowsy driving while showcasing the potential for technological innovation in automotive safety solutions.

Contents

1 Introduction **1**

1.1 Overveiw of Real-time Driver Drowsiness Detection 1

1.2 Motivation and Significance 1

1.3 Core Objectives and Features 2

1.4 Technological Innovations 2

1.5 Addressing the Challenges of Drowsy Driving 3

1.6 Ethical and Practical Considerations 3

1.7 Scope and Future Directions 3

2 Review of Literature **4**

2.1 DrowsyDetect: Android Application for Driver Drowsiness Detection Using Eye Closure Rate with Deep Learning 4

2.2 Driver-Drowsiness Detection System Using Deep Learning (CNN) 5

2.3 Driver Drowsiness Detection using Convolutional Neural Networks(CNNs) 5

2.4 Driver's Drowsiness Detection using MobileNet on Web 6

2.5 Driver Drowsiness Detection Using Machine Learning 7

2.6 Driver Drowsiness Detection Model Using Convolutional Neural Networks Techniques for Android Application 8

2.7 Smart Real-Time Video Surveillance Platform for Drowsiness Detection Based on Eyelid Closure 9

3 Project Vision **10**

3.1 Problem Statement 10

3.2 Business Opportunity 10

3.3 Objectives 11

3.4 Project Scope . . . 11
3.5 Constraints . . . 11
3.6 Stakeholders Description . . . 12

4 Software Requirements Specifications **14**
4.1 List of Features . . . 14
4.2 Functional Requirements . . . 15
4.3 Quality Attributes . . . 15
4.4 Non-Functional Requirements . . . 16
4.5 UML Diagrams . . . 16
4.5.1 Use Case Diagram . . . 16
4.5.2 Activity Diagram . . . 18
4.5.3 Architecture Diagram . . . 19

5 Iteration 1: UI Screens Design **21**
5.1 UI Design Overview . . . 21
5.2 Tools . . . 21
5.3 Screens . . . 22

6 Iteration 2 : FYP 1 Final **32**
6.1 Code and Data Collection . . . 32
6.2 Code Structure . . . 32
6.2.1 Directory Layout . . . 32
6.2.2 Key Flutter Components . . . 34
6.2.3 Future Work: Backend and Model Integration . . . 34
6.3 Data Collection . . . 35
6.3.1 Image Data . . . 35
6.3.2 Sensor Data . . . 37
6.3.3 Future Work: Data Analysis and Model Training . . . 37

7 Iteration 3 **38**
7.1 Model Training and Optimization . . . 38
7.2 Introduction . . . 38

7.3 Models Used . . . 38
7.3.1 Image Classification Models . . . 38
7.3.2 Sensor Data Models . . . 39
7.4 Model Architectures and Training Details . . . 39
7.4.1 MobileNet Architecture for Image Classification . . . 39
7.4.2 ConvLSTM2D Architecture for Sensor Data . . . 40
7.5 Challenges Faced . . . 41
7.6 Conclusion . . . 41

8 Iteration 4 : FYP 2 Final **42**
8.1 Backend System . . . 42
8.2 Code Implementation . . . 42
8.3 Backend Workflow . . . 45
8.4 Challenges and Future Improvements . . . 45

9 Implementation Details **46**
9.1 Programming Languages and Technologies . . . 46
9.1.1 Frontend . . . 46
9.1.2 Backend . . . 46
9.2 System Architecture . . . 46
9.3 Database . . . 47
9.4 Integration of Drowsiness Detection Module . . . 47
9.5 Error Handling and Validation . . . 48
9.6 Security Considerations . . . 48
9.7 User Experience (UX) Considerations . . . 48
9.8 Version Control . . . 48

10 Testing and Evaluation **49**
10.1 Overview . . . 49
10.2 API Testing . . . 49
10.3 Model Performance Testing . . . 51
10.4 Real-Time System Evaluation . . . 53

11 User Manual **54**

11.1 Overview . . . 54

11.2 Getting Started . . . 54

11.2.1 Account Setup . . . 54

11.3 Using the System . . . 55

11.3.1 Real-Time Drowsiness Detection . . . 55

11.3.2 Image-Based Detection . . . 55

11.3.3 Sensor-Based Detection . . . 55

11.4 Dashboard Features . . . 55

11.4.1 Accessing and Managing Results . . . 55

11.4.2 System Settings . . . 56

11.5 User Tips and Best Practices . . . 56

11.5.1 Ensuring Optimal Performance . . . 56

11.5.2 Data Privacy . . . 56

11.6 Troubleshooting . . . 57

11.6.1 Common Issues . . . 57

11.6.2 Support . . . 57

11.7 Conclusion . . . 57

12 Conclusions and Future Work **58**

12.1 Goals Achieved . . . 58

12.1.1 Development of a Functional System . . . 58

12.1.2 Real-Time Monitoring and Notifications . . . 58

12.1.3 User-Friendly Interface . . . 59

12.2 Future Work . . . 59

12.2.1 Advanced Model Optimization and Edge Deployment . . . 59

12.2.2 Expanded Detection Capabilities . . . 59

12.2.3 Integration with Vehicle Systems . . . 60

12.2.4 Improved Data Collection and Augmentation . . . 60

12.2.5 Field Testing and Deployment . . . 60

12.2.6 Collaboration with Industry Partners . . . 60

12.3 Conclusion 61

References **63**

List of Figures

4.1 Use Case Diagram 17
4.2 Activity Diagram: Driver Drowsiness Detection Process 18
4.3 Architecture Diagram of the Drowsiness Detection System (Mobile) . . . 19
4.4 Architecture Diagram of the Drowsiness Detection System (Computer) . . 20

5.1 Driver Drowsiness Detection (DDD) Loading Screen 22
5.2 DDD Sign-Up Screen 23
5.3 DDD Login Screen 24
5.4 DDD Forgot Password Screen 25
5.5 DDD Main Screen 26
5.6 DDD Lookup Emergency Contacts Screen 27
5.7 DDD Start Trip Screen 28
5.8 DDD Alert Screen 29
5.9 DDD User Account Screen 30
5.10 DDD Settings Screen 31

6.1 Flutter Project Directory Structure 33
6.2 Open Eyes 36
6.3 Closed Eyes 36
6.4 Yawning 36
6.5 No Yawning 36
6.6 Representative Samples from the Image Dataset. The dataset includes labeled examples for detecting drowsiness-related states such as open/-closed eyes and yawning behaviors. 36

List of Tables

6.1 Sample Sensor Data for Driving Behavior Classification 37

10.1 API Test Cases 50

10.2 Sample Sensor Data for Driving Behavior Classification 52

Chapter 1

Introduction

1.1 Overveiw of Real-time Driver Drowsiness Detection

Road accidents remain a leading cause of fatalities worldwide, with driver fatigue being a significant contributing factor. The risks associated with drowsy driving are particularly severe for long-route drivers, who often experience extended periods of reduced alertness. The project, titled "Real-time Image & Sensor-based Driver Drowsiness Detection using Deep Learning," addresses this challenge by integrating advanced deep learning techniques with mobile technologies to create a real-time system capable of detecting and mitigating driver drowsiness effectively.

1.2 Motivation and Significance

The motivation for this project stems from the increasing demand for intelligent driver assistance systems that proactively reduce accident risks. Conventional solutions such as scheduled rest breaks or manual monitoring are often inadequate in real-world scenarios. The integration of image-based analysis and mobile sensor data offers a promising approach to overcoming these limitations. By developing an efficient and scalable detection system, this project aims to contribute to road safety and public well-being.

1.3 Core Objectives and Features

The primary goal of this project is to design and implement a real-time driver drowsiness detection system that combines image analysis with mobile sensor data. Key objectives include:

Real-time Drowsiness Detection: Using MobileNet, a lightweight convolutional neural network, to classify eye states from real-time webcam images, identifying signs of drowsiness such as prolonged eye closure.

Behavioral Analysis: Employing a CNN-LSTM model to process gyroscope and accelerometer data from mobile sensors, detecting patterns indicative of aggressive or erratic driving behavior.

Alert System: Developing a responsive alert mechanism to notify drivers of drowsiness or risky behavior, potentially preventing accidents.

Cross-platform Accessibility: Creating a mobile application using Flutter, ensuring a user-friendly interface and compatibility across Android and iOS devices.

Integration of TensorFlow Lite: Enabling the deployment of deep learning models on mobile devices for real-time performance with minimal computational overhead.

1.4 Technological Innovations

This project demonstrates the application of advanced technologies to solve a critical real-world problem. Key innovations include:

MobileNet for Image Processing: A lightweight convolutional neural network optimized for real-time performance, allowing accurate eye-state classification. CNN-LSTM for Temporal Analysis: A hybrid model combining convolutional layers with long short-term memory (LSTM) units, effectively capturing temporal patterns in sensor data. Flutter-based Application Development: Ensuring seamless integration of functionalities into a cross-platform mobile application for end-users. TensorFlow Lite Integration: Enabling

efficient execution of deep learning models on resource-constrained mobile devices.

1.5 Addressing the Challenges of Drowsy Driving

This project seeks to overcome several challenges inherent in drowsy driving detection:

Real-time Performance: Ensuring the system operates with minimal latency, providing timely alerts. Cost-effectiveness: Designing a solution that is accessible and does not rely on expensive hardware, making it suitable for widespread adoption. Scalability: Enabling easy integration into existing driver assistance systems or standalone deployment on mobile devices. Driver Safety and Usability: Minimizing distraction while ensuring the system is intuitive and easy to use.

1.6 Ethical and Practical Considerations

Developing a real-time drowsiness detection system involves addressing ethical and practical challenges, including:

Privacy Protection: Ensuring that driver images and sensor data are securely processed and stored. False Alerts: Minimizing false positives and negatives to maintain user trust and system reliability. Inclusivity: Designing the system to accommodate diverse user needs, including varying driving conditions and habits. Safety Assurance: Ensuring the system enhances rather than hinders the driver's ability to operate the vehicle safely.

1.7 Scope and Future Directions

The Real-time Image & Sensor-based Driver Drowsiness Detection project is designed to be a scalable and versatile solution for detecting driver fatigue and unsafe behavior. While its immediate application focuses on road safety, its methodologies and findings have broader implications for intelligent transportation systems and human-computer interaction.

Chapter 2

Review of Literature

2.1 DrowsyDetect: Android Application for Driver Drowsiness Detection Using Eye Closure Rate with Deep Learning

Year: 2023

Basic Idea: The paper introduces DrowsyDetect, an Android app utilizing deep learning (MobileNet, VGG16, custom CNN) to detect driver drowsiness by analyzing eye closure rate.

Methodologies: Deep learning models (MobileNet, VGG16, custom CNN) were used for drowsiness detection, employing Haar Cascade for feature extraction to detect faces and eyes. Data preprocessing involved normalization and serialization using the pickle library. Models were trained for 15 epochs with specific hyperparameters like a learning rate of 0.1, binary cross-entropy loss, Adam optimizer, and Sigmoid activation function.

Results: MobileNet and VGG16 both achieved 100% accuracy, with VGG16 reaching this after 615 epochs. The custom CNN model attained an accuracy of 98.75% after 1015 epochs. Comparatively, MobileNet and VGG16 achieved 100% accuracy, while the custom CNN model reached 98.75%.

Limitations: The study didn't explore integrating complex features like sleep schedules

or emergency triggers. It focused solely on eye closure rate and had a limited dataset, potentially impacting model generalizability. Challenges with Haar Cascade for feature extraction weren't discussed.

2.2 Driver-Drowsiness Detection System Using Deep Learning (CNN)

Year: 2023

Basic Idea: The paper presents a real-time driver drowsiness detection system using CNNs, integrating eye blink analysis and yawning detection. It involves video extraction, preprocessing, and CNN-based classification.

Methodologies: The system employed the SSD MobileNet V2 FPNLite 320x320 pre-trained model, optimized for speed and efficiency in drowsiness detection. Training involved a dataset of yawning and blinking images to enhance accuracy.

Results: The deep learning model achieved a drowsiness detection accuracy of 62-87%. Its mean Average Precision (mAP) was 0.595 at 5000 training steps and 0.56 at 7000 training steps. Additionally, the average recall (AR100(Large)) was 0.686 at 5000 training steps and 0.662 at 7000 training steps.

Limitations: The User Acceptance Testing (UAT) involved only a small sample size of seven participants, which could limit generalizability.

2.3 Driver Drowsiness Detection using Convolutional Neural Networks(CNNs)

Year: 2023

Basic Idea: The paper presents a real-time driver drowsiness detection system using CNNs, incorporating eye blink analysis and yawning detection. Steps involve video extraction, preprocessing, data augmentation, and CNN-based classification.

Methodologies: Convolutional Neural Networks (CNNs) were used for driver drowsiness

detection, integrating eye blink analysis and yawning detection. A specific CNN model with 3 convolution layers and ReLU activation functions was employed for yawning feature extraction. The architecture included Conv2D and MaxPooling layers for efficient image classification.

Results: The proposed Convolutional Neural Network model achieved a 95% accuracy for driver drowsiness detection. Additionally, the CNN algorithm demonstrated superior accuracy rates of 99% compared to existing CNN-based methods for recognizing driver fatigue conditions. Evaluation metrics for yawning detection included precision at 0.9207, recall at 0.8774, and F1 score at 0.8985.

Limitations: The study lacks information on how well the proposed model would generalize to different datasets and real-world scenarios, indicating a need for further validation. Specifically, more testing is needed to assess the model's performance across diverse driving conditions and with a larger dataset.

2.4 Driver's Drowsiness Detection using MobileNet on Web

Year: 2023

Basic Idea: The study focuses on real-time drowsiness detection using MobileNet, a deep convolutional neural network. MobileNet's design prioritizes efficiency for mobile devices, making it suitable for applications like real-time drowsiness detection.

Methodologies: The study employed facial feature extraction and classification for drowsiness detection, analyzing data collected from cameras monitoring the driver's face using machine learning algorithms. Specifically, a combination of Haar-like features and the AdaBoost algorithm was used for eye state analysis, while head pose analysis utilized a particle filter-based algorithm. Drowsiness detection was conducted using the MobileNet V1 architecture.

Results: The drowsiness detection system, utilizing the MobileNet V1 architecture, achieved an impressive accuracy of 97.45%. Overall, the system exhibited high accuracy in detecting driver sleepiness, with an accuracy rate of 92.6%. Additionally, the

proposed method for eye state analysis achieved a notable accuracy of 94.23% in detecting drowsiness.

Limitations: The study concentrated on drowsiness detection using MobileNet without exploring other deep learning architectures for comparison. It emphasized facial feature extraction and classification, potentially neglecting other physiological indicators of drowsiness. Furthermore, the system's performance evaluation was based on specific datasets and conditions, possibly restricting its generalizability to diverse real-world scenarios.

2.5 Driver Drowsiness Detection Using Machine Learning

Year: 2023

Basic Idea: The paper introduces a real-time system using computer vision and machine learning to detect driver drowsiness. It aims to address road safety concerns, especially in India.

Methodologies: The system employs computer vision, machine learning, and sensor integration for drowsiness detection. During training, data preprocessing techniques are applied, and a Convolutional Neural Network (CNN) model is used for prediction. The architecture integrates a CNN-based machine learning algorithm for sleepiness detection and a user-friendly interface.

Results: The developed drowsiness detection application achieved an F1 score of 89%, demonstrating balanced performance in precision and recall. It outperformed existing solutions in accuracy, precision, recall rates, and overall F1 score, indicating its effectiveness in identifying drowsiness levels and minimizing false positives.

Limitations: There's a lack of extensive investigations into incorporating machine learning algorithms into user-friendly mobile applications customized for specific geographical locations like India. Additionally, there's a scarcity of user studies assessing the usability, efficacy, and real-world implications of the drowsiness detection application. Further research is needed to authenticate the practical usefulness of the developed application in

real-world scenarios.

2.6 Driver Drowsiness Detection Model Using Convolutional Neural Networks Techniques for Android Application

Year: 2020

Basic Idea: The paper focuses on real-time driver drowsiness detection using a lightweight neural network approach. It analyzes facial landmarks captured by a camera and processed by a Convolutional Neural Network (CNN) for classification.

Methodologies: The paper utilized facial landmarks detected by a camera and processed them through a Convolutional Neural Network (CNN) for drowsiness classification. Video preprocessing involved extracting images from video frames in 4 stages, incorporating MaxPooling and ReLU functions. The system detected facial landmarks from mobile device images and fed them into a CNN-based trained Deep Learning model for detecting drowsy driving behavior.

Results: The D2CNN-FLD method achieved an 83.3% overall accuracy in detecting driver drowsiness. Compared to the older D2MLP-FLD model, it displayed significant reductions in size, complexity, and storage, with a maximum size of 75 KB. In terms of drowsiness detection time, D2CNN-FLD achieved 38.45 ms on a Dell workstation with NVIDIA Quadro P4000.

Limitations: The technology may face limitations when facial features are obstructed by sunglasses or under poor lighting conditions. Improvement opportunities exist in enhancing facial feature detection, particularly in challenging lighting environments.

2.7 Smart Real-Time Video Surveillance Platform for Drowsiness Detection Based on Eyelid Closure

Year: 2019

Basic Idea: The paper introduces a method for real-time drowsiness detection in surveillance videos by analyzing eyelid closure using face detection and eyelid curvature detection. It involves detecting faces with the Viola-Jones algorithm, identifying the driver's face, and analyzing the eye region for eyelid curvature. Closed eyelids are detected based on concave curves, with upward curves indicating closure and downward curves indicating open eyes.

Methodologies: The Viola-Jones object detection algorithm was used for face detection, followed by extracting the eyes for further analysis. Eyelid curvature was determined using a modified Sobel operator to detect eyelid closure, and eyes were classified as open or closed based on this curvature.

Results: The method achieved up to 95% accuracy on a uniform background dataset, 70% on a benchmark dataset with varied face deformations, and maintained 95% accuracy in real-time video datasets. Eyelid curvature detection and eye classification had a 97% accuracy relative to eye detection. While correct eye classification was observed in the benchmark dataset, some incorrect results occurred. Overall, the method effectively detected open and closed eyes.

Limitations: Dataset limitations stemming from the rejection of images with severe out-of-plane and in-plane rotations impact the dataset's representativeness. The accuracy of the proposed method relies on eye detection accuracy, which was 98%, thus influencing the overall relative accuracy.

Chapter 3

Project Vision

3.1 Problem Statement

Driver drowsiness poses a critical threat to road safety, particularly during long-distance travel where drivers are more likely to experience fatigue. This condition significantly increases the likelihood of accidents, endangering not only the drivers themselves but also passengers, pedestrians, and other road users. Existing drowsiness detection systems often suffer from limitations such as the lack of real-time capabilities, complex user interfaces, or high implementation costs, which reduce their effectiveness and accessibility. These challenges underscore the urgent need for a solution that is both efficient and user-friendly, capable of providing timely interventions to prevent potential road mishaps.

3.2 Business Opportunity

The rising awareness of road safety and the rapid advancement of automotive technologies present a substantial business opportunity in the domain of intelligent safety systems. A robust and reliable real-time drowsiness detection system can cater to the increasing demand for innovative safety solutions in the automotive sector. This project aims to capitalize on this demand by developing a cost-effective, scalable, and user-centric system. Such a solution has the potential to attract transportation companies, commercial fleet op-

erators, and individual drivers, while simultaneously contributing to societal well-being by reducing accident rates caused by drowsy driving.

3.3 Objectives

The primary objective of this project is to design and develop a real-time drowsiness detection system integrated into a mobile application. This application will leverage state-of-the-art deep learning techniques to monitor driver behavior and issue timely alerts, ensuring enhanced road safety. Additionally, the project aims to create a user-friendly interface accessible to drivers of varying technological proficiencies, fostering broad adoption across diverse user groups. Ultimately, this initiative seeks to merge technological innovation with practical functionality, delivering a solution that aligns with user needs and industry standards.

3.4 Project Scope

The scope of this project encompasses the end-to-end development of a mobile application for detecting driver drowsiness in real-time. It includes the collection and preprocessing of image and sensor data, the design and training of deep learning models, and the seamless integration of these models into a cross-platform mobile application. Furthermore, the scope extends to rigorous testing and optimization of the system to ensure its reliability, accuracy, and responsiveness in diverse real-world scenarios. The project's deliverables will include a fully functional prototype capable of providing timely drowsiness alerts, with potential for future enhancements and scalability.

3.5 Constraints

Despite the project's potential, several constraints must be addressed to ensure its successful implementation. Limited resources, including budget, time, and personnel, pose

challenges in developing and testing the system comprehensively. The reliance on existing technologies and frameworks, such as MobileNet for image processing and CNN-LSTM for sensor data analysis, necessitates careful integration and optimization. Additionally, the project must adhere to regulatory standards and guidelines governing mobile applications and road safety systems, which may impose technical and operational constraints. Addressing these challenges will require innovative problem-solving and efficient resource management.

3.6 Stakeholders Description

The success of this project depends on the collaboration and alignment of interests among key stakeholders.

End Users represent a critical stakeholder group, including long-distance drivers, commercial transportation operators, and fleet management companies. Their primary goal is to ensure safer driving experiences by receiving accurate and timely alerts to mitigate drowsiness-related risks. However, they currently face the challenge of limited access to effective and intuitive drowsiness detection systems.

Developers, comprising the project team, are tasked with designing, developing, and deploying a reliable application. Their goal is to create a robust system capable of real-time performance while addressing challenges such as resource limitations and the complexity of integrating deep learning models into mobile platforms.

Regulatory Authorities oversee compliance with road safety standards and ensure that the system meets stringent reliability and effectiveness benchmarks. They play a vital role in certifying the application's adherence to established guidelines, which is crucial for its acceptance and deployment in the market.

Investors are another pivotal stakeholder group, as their financial support drives the project's development and implementation. Their primary goal is to achieve a return on investment by ensuring the project's alignment with market demands and trends. They are particularly concerned with the viability, scalability, and market positioning of the

system.

By addressing the goals and concerns of these diverse stakeholders, the project seeks to deliver a solution that is not only technologically advanced but also practical, compliant, and commercially viable.

Chapter 4

Software Requirements Specifications

4.1 List of Features

The proposed system incorporates the following key features to provide an effective and user-friendly solution for real-time driver drowsiness detection:

- **Real-Time Drowsiness Detection**: Utilization of advanced deep learning algorithms to monitor and analyze driver behavior in real-time.
- **User-Friendly Interface**: A mobile application designed with an intuitive and accessible interface for seamless interaction.
- **Alert System**: Immediate notifications to alert drivers when drowsiness is detected, ensuring timely interventions to prevent accidents.
- **Data Collection and Preprocessing**: Capabilities to capture, preprocess, and analyze real-time data, including images and sensor input.
- **Cross-Platform Integration**: Compatibility with both iOS and Android platforms, ensuring wide accessibility for users.

4.2 Functional Requirements

The system is designed to meet the following functional requirements, ensuring effective performance and usability:

- The system shall capture real-time images or video feed from the driver's device camera.
- The system shall analyze facial features and eye movements to detect signs of drowsiness.
- The system shall generate alerts in real-time when drowsiness is detected.
- The application shall have options for customization, including alert thresholds and notification preferences.

4.3 Quality Attributes

The system is designed to uphold the following quality attributes to ensure a superior user experience and dependable performance:

- **Accuracy**: The system shall accurately detect drowsiness with a high degree of precision.
- **Reliability**: The system shall operate consistently under various conditions, providing reliable alerts to users.
- **Efficiency**: The system shall process data efficiently, minimizing processing delays and resource consumption.
- **Usability**: The application interface shall be intuitive and easy to navigate for users of all skill levels.

4.4 Non-Functional Requirements

To maintain high standards of performance and security, the system shall meet the following non-functional requirements:

- **Performance**: The application shall process data and generate alerts in real-time, with minimal latency to ensure immediate driver response.
- **Security**: User data, including images and sensor inputs, shall be safeguarded using robust encryption and security protocols, ensuring privacy and compliance with data protection standards.
- **Compatibility**: The application shall be fully functional across a wide range of mobile devices and operating systems, including varying hardware capabilities.
- **Scalability**: The system architecture shall support future upgrades and extensions, accommodating additional features and increased user demand without significant redesign.

4.5 UML Diagrams

In this section, we present the various UML diagrams that define the system's behavior, structure, and interactions. These diagrams include the Use Case Diagram, Activity Diagrams, Architecture Diagram, and Sequence Diagram. Each diagram provides a visual representation of the system's functionality and architecture.

4.5.1 Use Case Diagram

The Use Case Diagram represents the functional interactions between the system and its users, illustrating the main functions and the actors that interact with them.

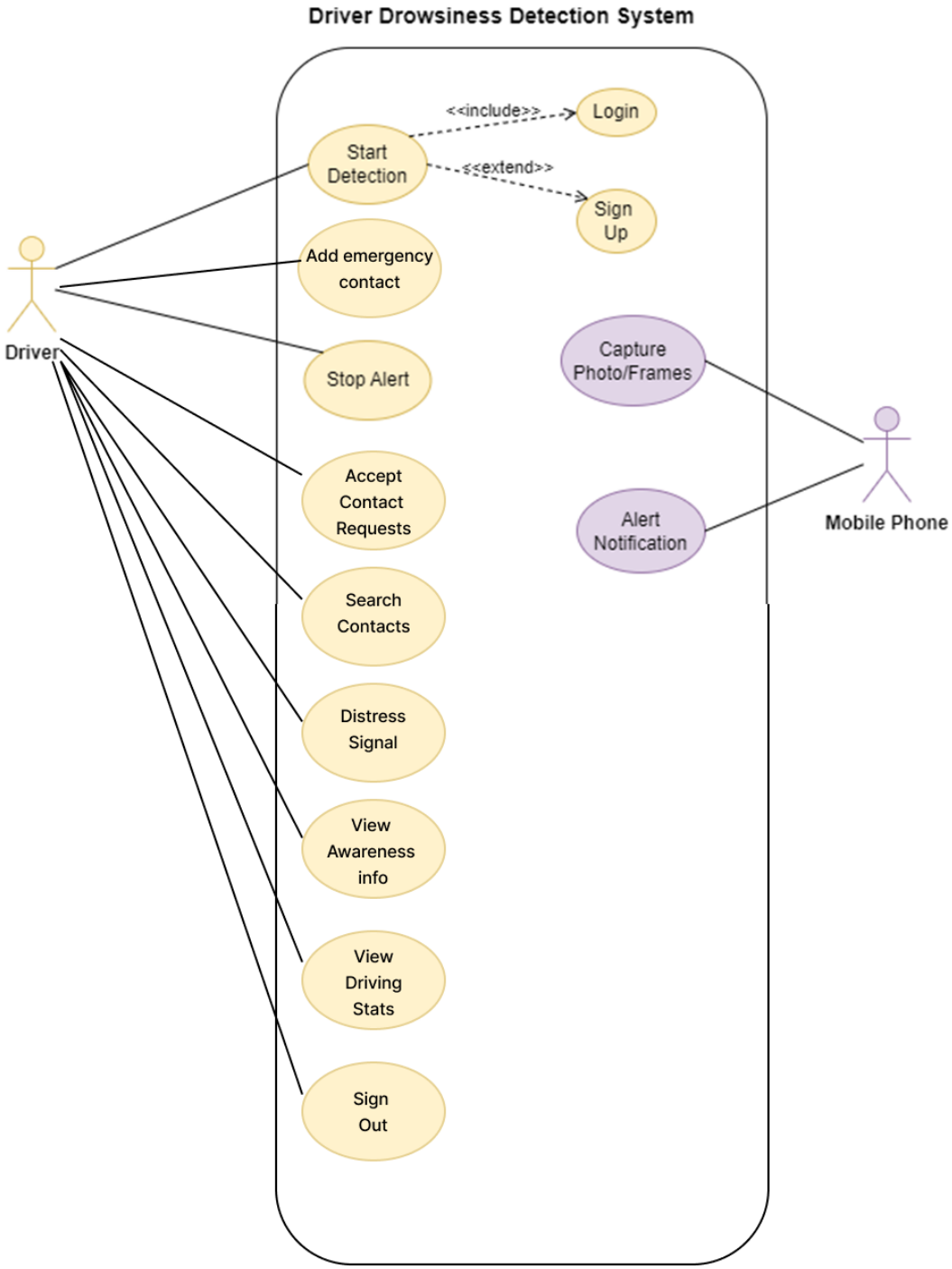


Figure 4.1: Use Case Diagram

4.5.2 Activity Diagram

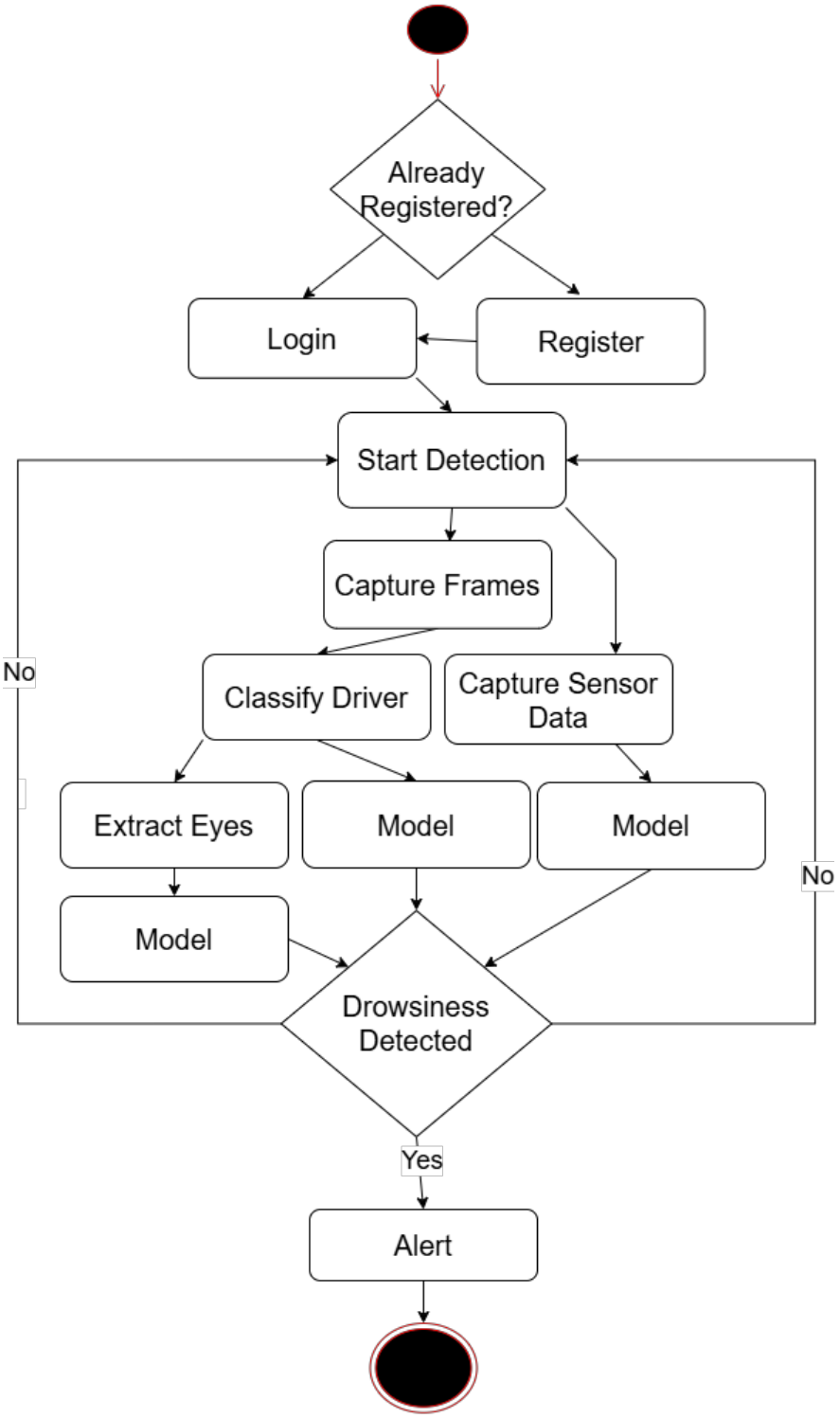


Figure 4.2: Activity Diagram: Driver Drowsiness Detection Process

4.5.3 Architecture Diagram

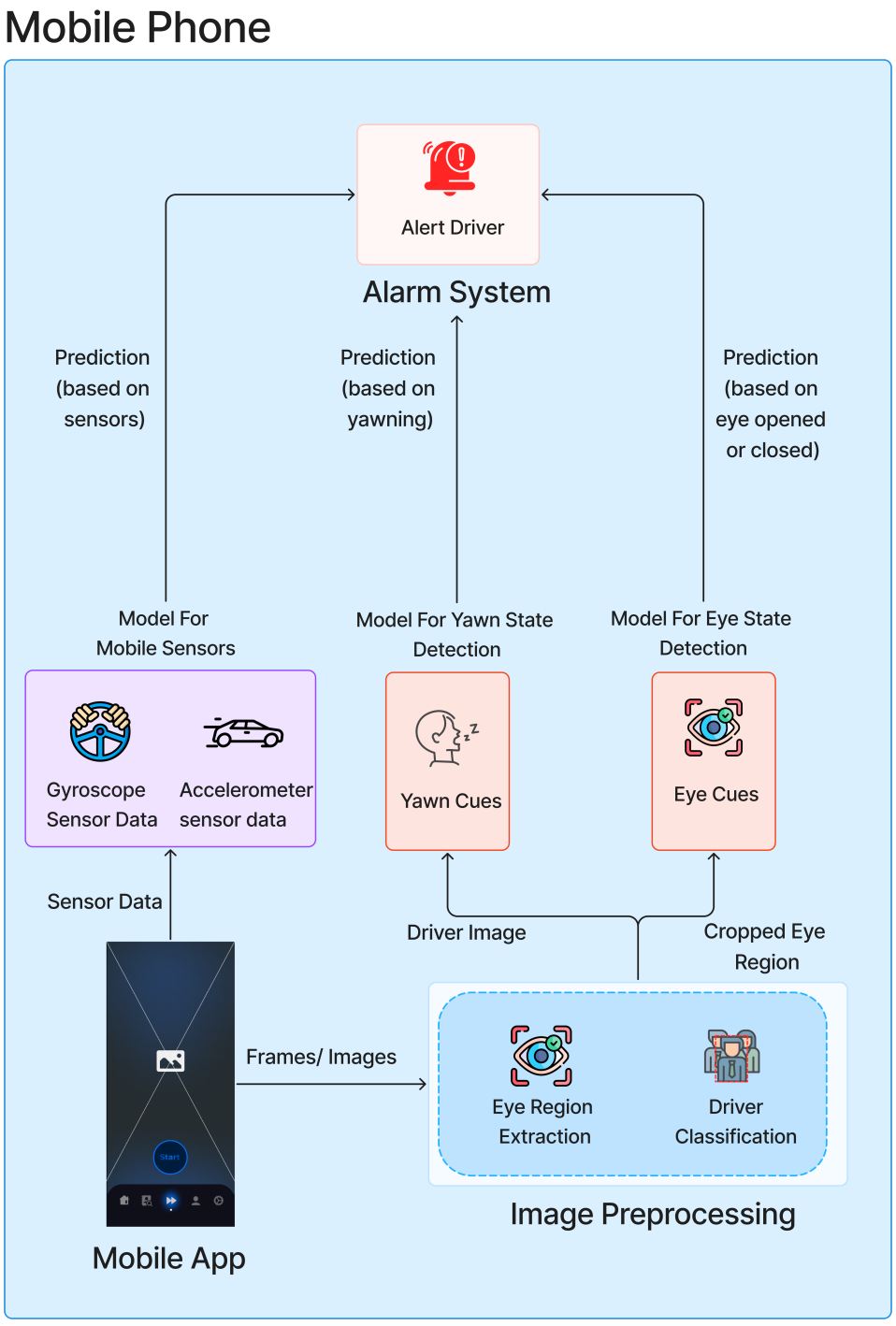


Figure 4.3: Architecture Diagram of the Drowsiness Detection System (Mobile)

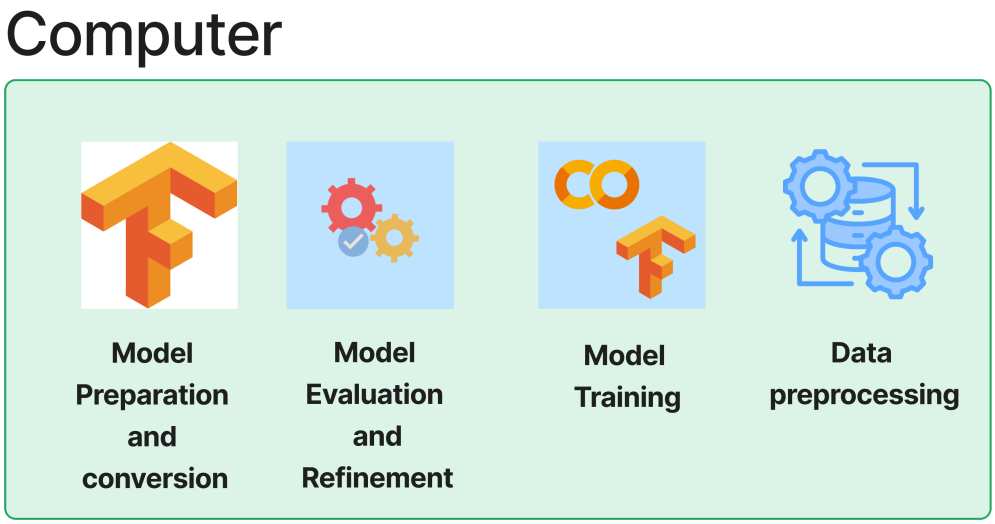


Figure 4.4: Architecture Diagram of the Drowsiness Detection System (Computer)

Chapter 5

Iteration 1: UI Screens Design

5.1 UI Design Overview

Before implementing the core functionality of the Driver Drowsiness Detection (DDD) system, we focused on designing the user interface (UI). This allowed us to visualize how the application will look and how users will interact with the system. The UI design is intended to ensure a seamless, user-friendly experience while effectively presenting critical information related to driver safety.

5.2 Tools

The following tools were used for creating and visualizing the UI of the mobile application:

- Figma: To design and visualize the front end of the application.
- Sketch: Used for finalizing high-fidelity designs.

5.3 Screens

The UI screens are categorized based on the user journey and system functionality. Below are the different screens designed for the Driver Drowsiness Detection (DDD) app:

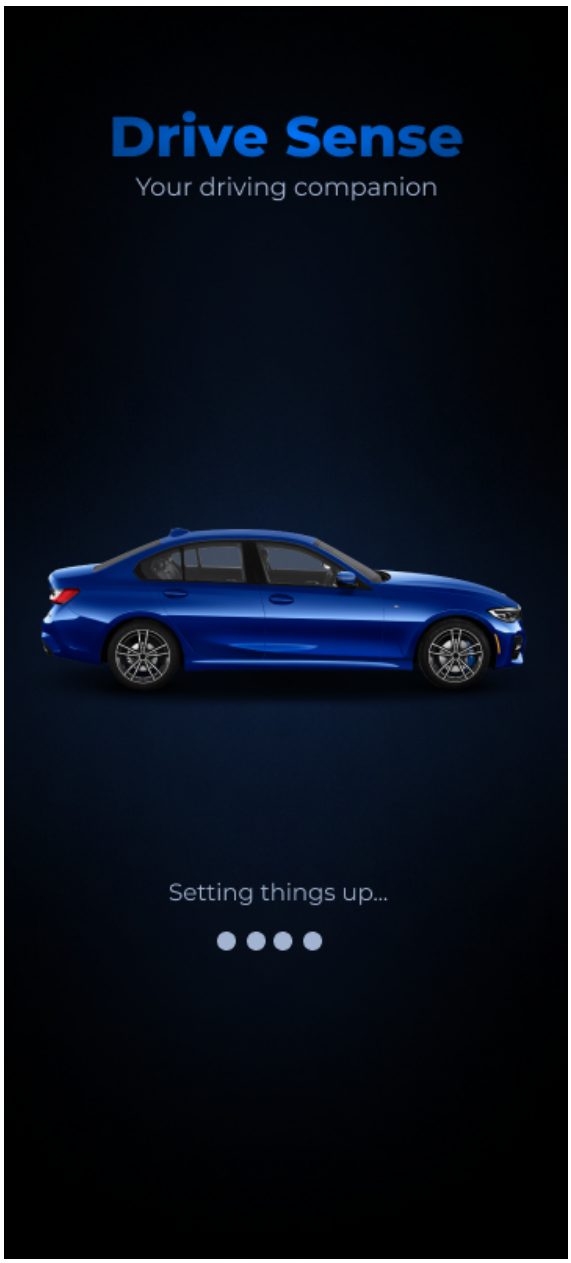


Figure 5.1: Loading Screen

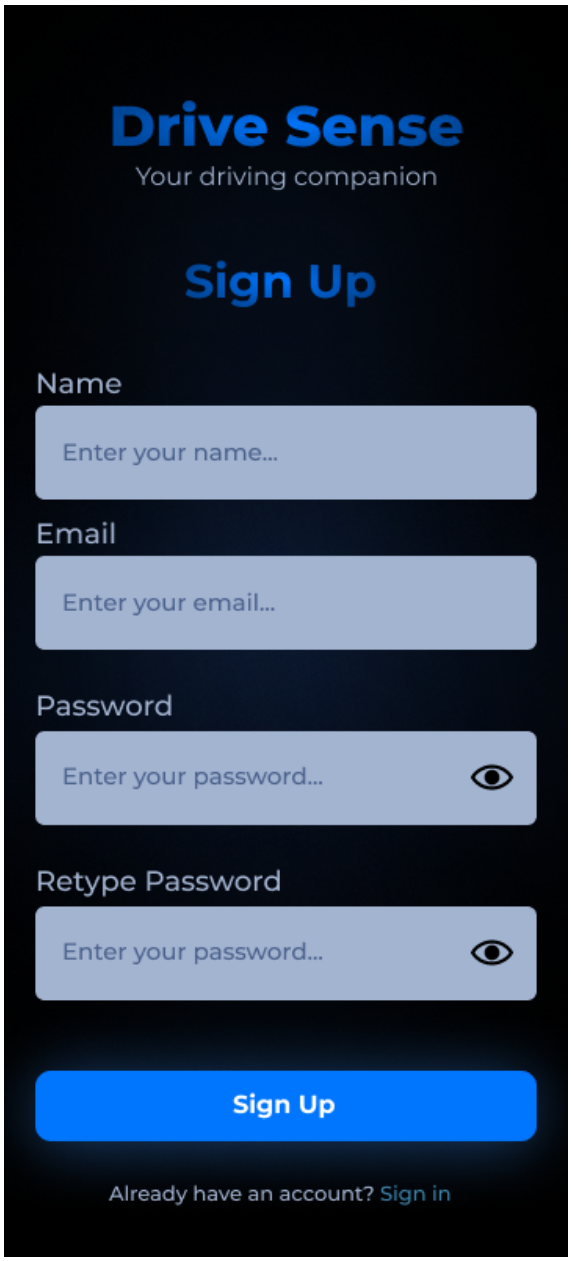


Figure 5.2: Sign-Up Screen

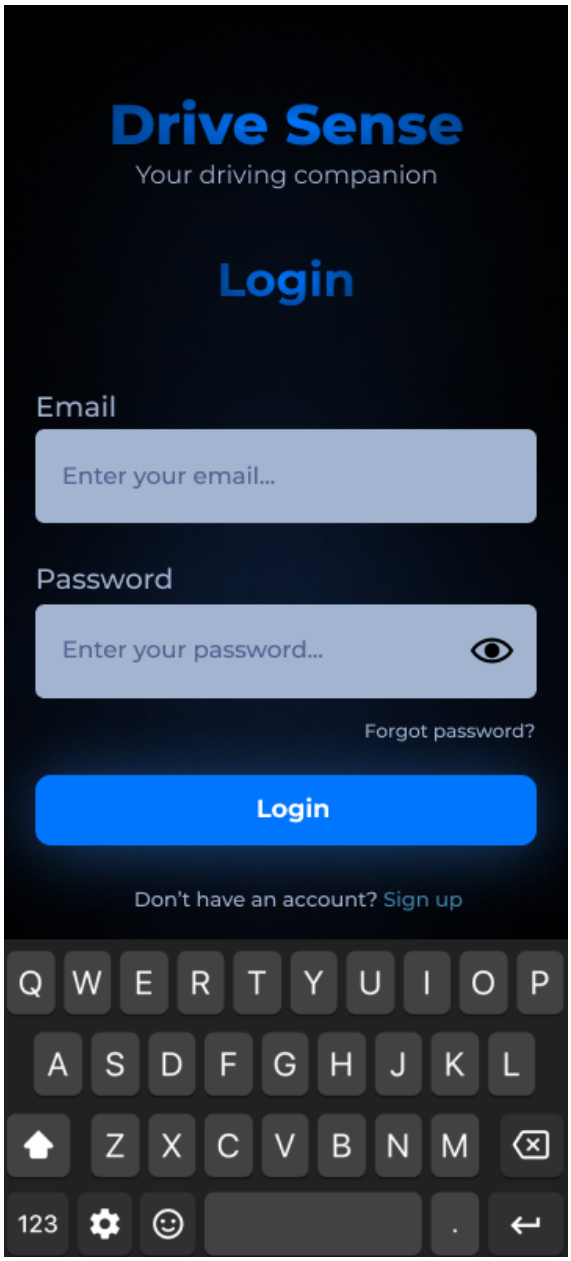


Figure 5.3: Login Screen

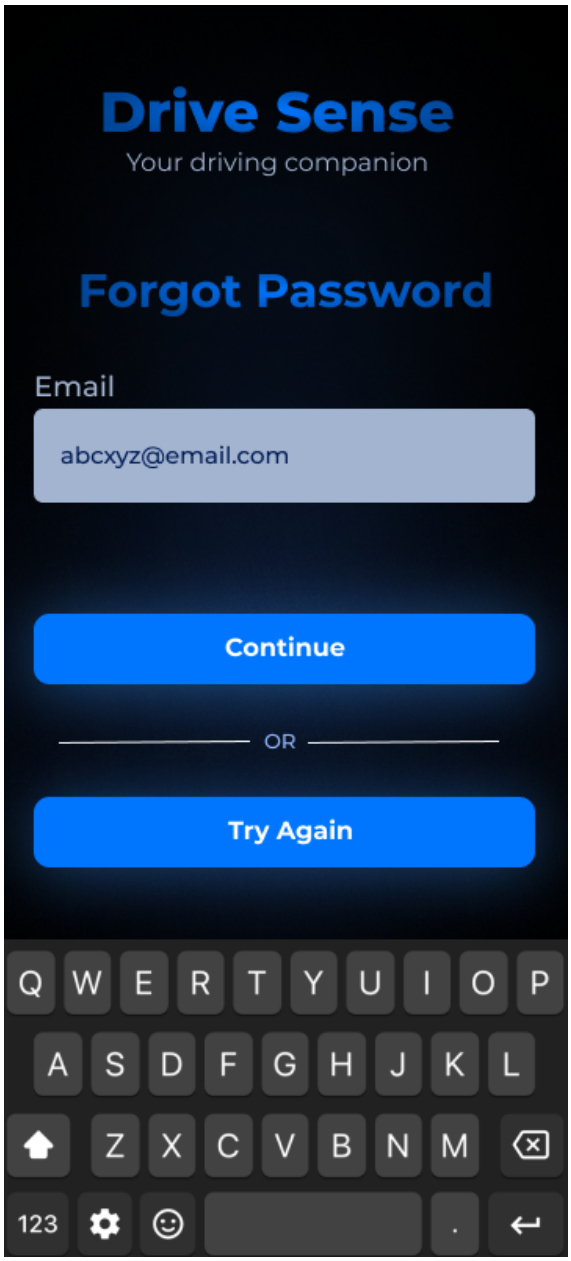


Figure 5.4: Forgot Password Screen

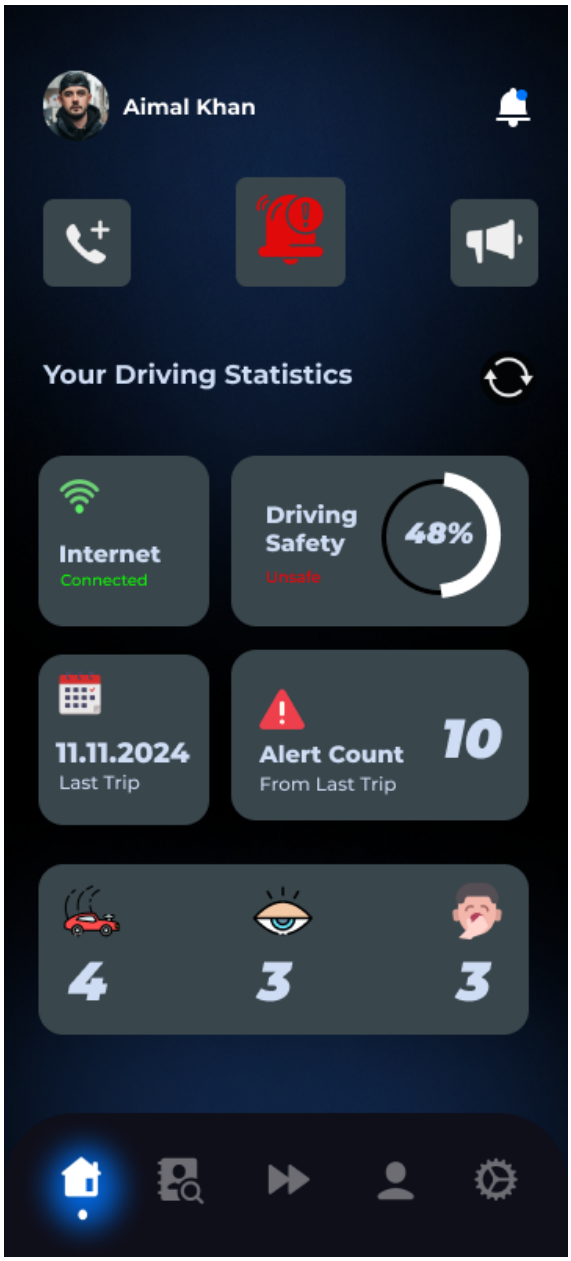


Figure 5.5: Home Screen

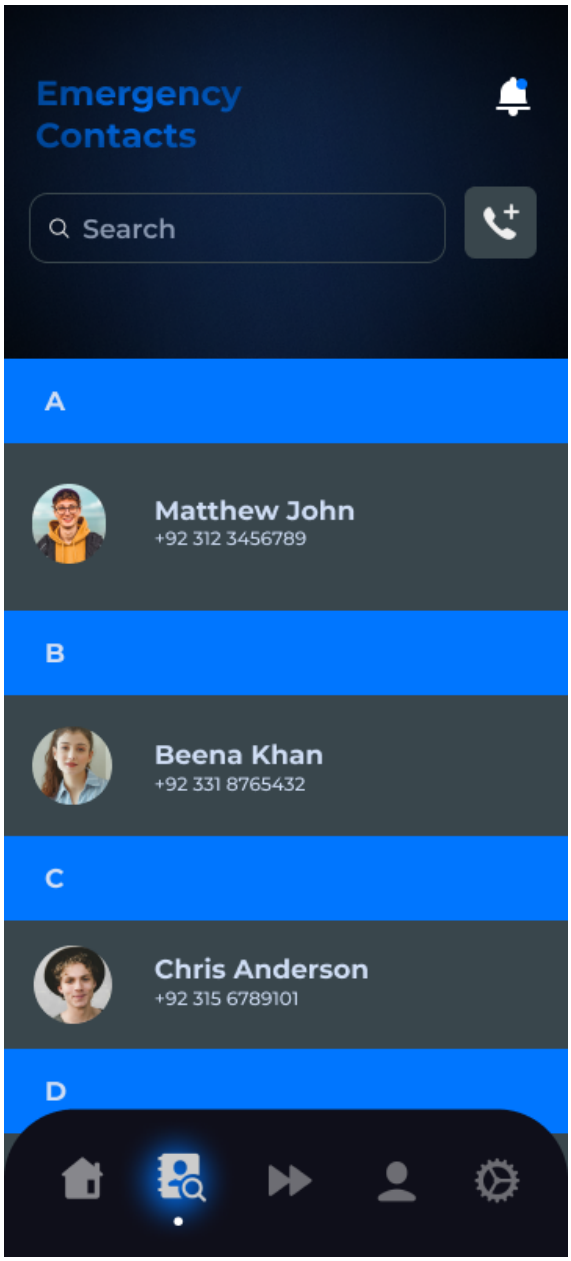


Figure 5.6: Contacts Screen

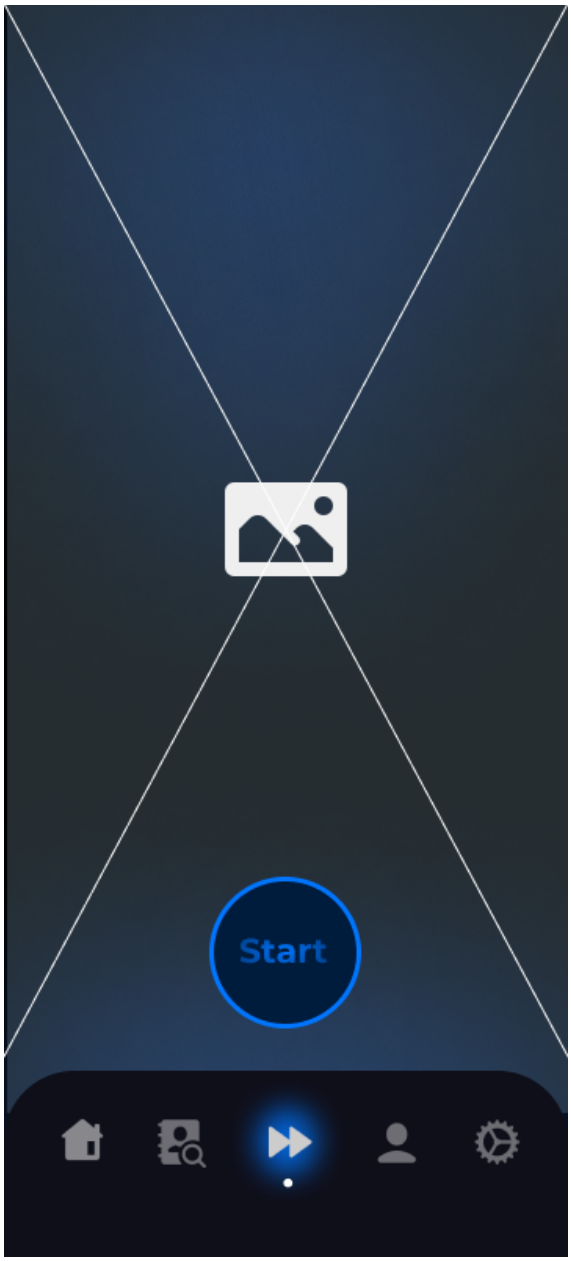


Figure 5.7: Start Trip Screen

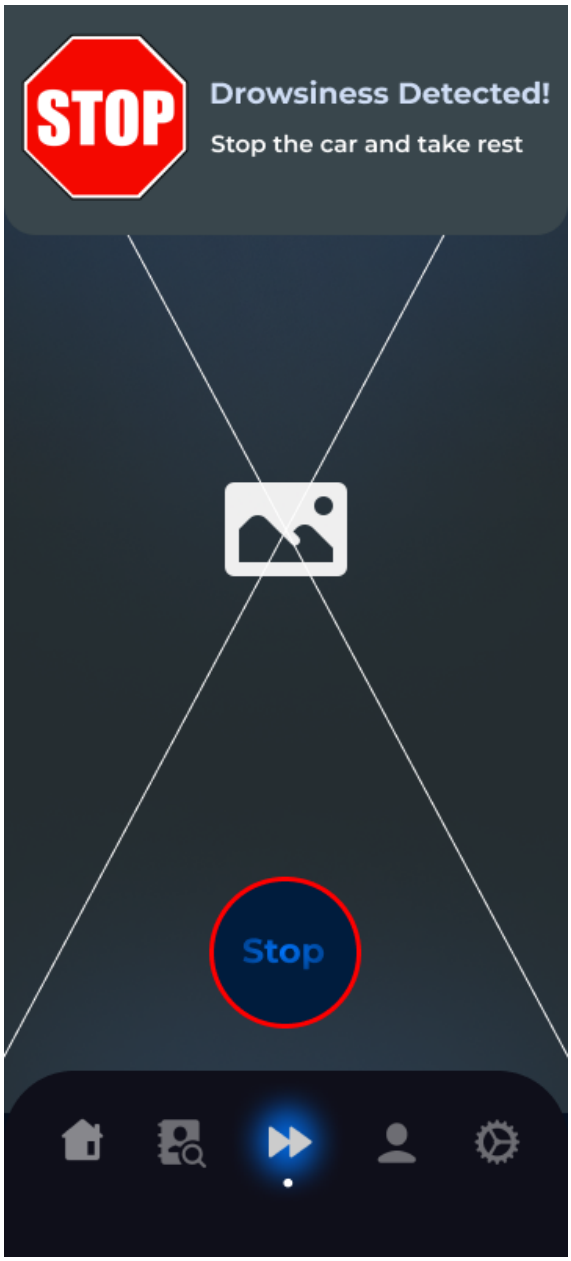


Figure 5.8: Alert Screen

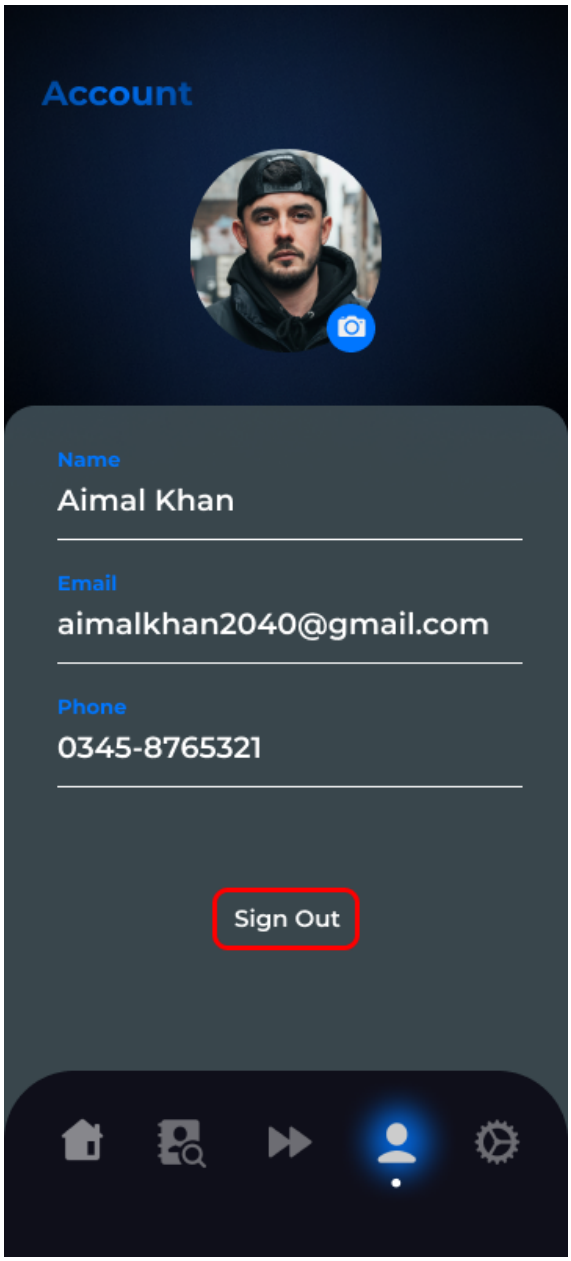


Figure 5.9: User Account Screen



Figure 5.10: Settings Screen

Chapter 6

Iteration 2 : FYP 1 Final

6.1 Code and Data Collection

The Driver Drowsiness Detection (DDD) project involves a combination of mobile application development, machine learning, and data collection. The first phase of this project focused on setting up the foundational elements, including gathering relevant datasets, structuring the mobile application, and defining workflows. The image and sensor data required for this project were carefully curated to ensure diversity and relevance. Additionally, the mobile application was built with scalability in mind to support real-time model integration in later stages. This chapter describes the data and code organization, providing an overview of the development process.

6.2 Code Structure

The mobile application for Driver Drowsiness Detection (DDD) is developed using Flutter. Below is an overview of the code structure and its key components.

6.2.1 Directory Layout

The directory structure of the application is shown in Figure 6.1.

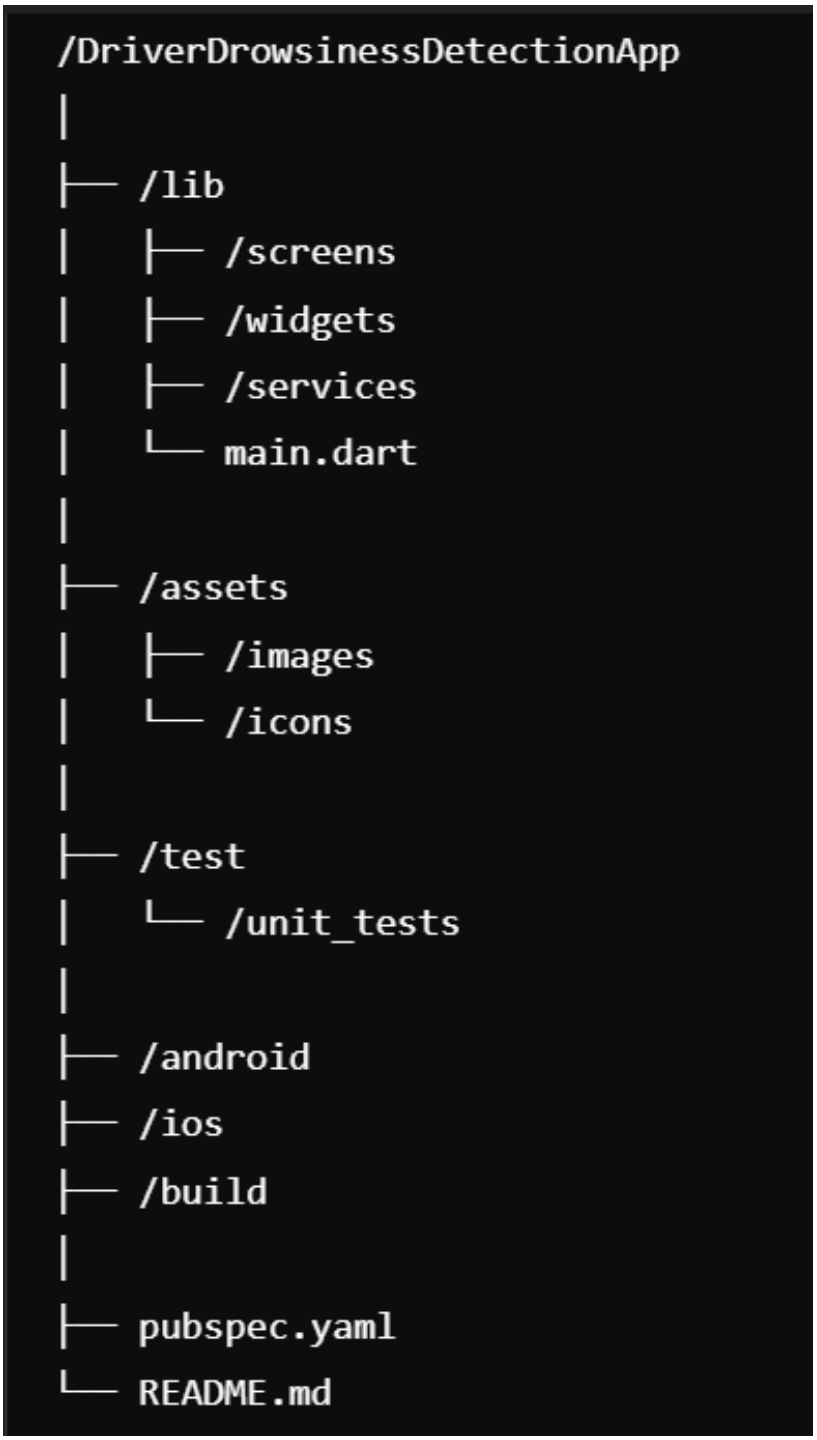


Figure 6.1: The directory structure of the Flutter project.

6.2.2 Key Flutter Components

The mobile application for the Driver Drowsiness Detection (DDD) system is built using Flutter, and it consists of several key components. The `Main.dart` file serves as the entry point of the application, defining the routes and navigation structure. The `Screens` represent the user interface for various features, such as login and trip initiation. Reusable `Widgets` are employed for common elements like buttons or alert dialogs, ensuring consistency across the application. Finally, the `Services` component acts as a placeholder for backend integration, which will be developed in the second phase of the project (FYP2). These components together form the structure that enables the application to function seamlessly and paves the way for future model integration and backend development.

6.2.3 Future Work: Backend and Model Integration

In FYP2, several key enhancements will be implemented to improve the functionality of the Driver Drowsiness Detection system. The backend will be developed to securely store and retrieve user data, enabling features such as trip history and the recording of drowsiness detection results. Additionally, real-time model integration will be a crucial development, allowing the app to analyze both image and sensor data to detect signs of driver drowsiness. To enhance the user experience and safety, an alert system will be implemented. This system will generate real-time warnings based on the model's predictions, notifying drivers of potential drowsiness. These enhancements will ensure that the app is not only functional but also scalable and responsive in real-time environments.

Key Enhancements in FYP2:

- **Backend Development:** Secure API integration for storing and retrieving user data, including trip history and detection results.
- **Model Integration:** Deployment of real-time drowsiness detection models for both image and sensor data.
- **Alert System:** Implementation of an alert system to notify drivers of potential drowsiness based on model predictions.

6.3 Data Collection

The datasets for the Driver Drowsiness Detection system were collected from various sources to ensure both the quality and diversity of the data. A significant portion of the image data was sourced from Kaggle, providing a variety of labeled samples specifically curated for training and testing purposes in the context of driver drowsiness detection. This dataset includes diverse images of drivers in various states of alertness, which allows for better model generalization. The sensor data, on the other hand, was recorded using gyroscope and accelerometer readings from a mobile device, which help in identifying different driving behaviors associated with drowsiness, such as erratic movement or lack of responsiveness. These datasets form the core foundation for training machine learning models capable of detecting drowsiness in real-time, utilizing both image-based and sensor-based input.

To ensure that the model can generalize well across different scenarios, the data was pre-processed and augmented where necessary. Data augmentation techniques, such as rotation and flipping for the image dataset, as well as noise addition for sensor data, were applied to improve the robustness of the models. The balance and variety within the datasets help ensure the system can accurately detect drowsiness across a wide range of real-world conditions.

6.3.1 Image Data

The image dataset comprises four key categories: open eyes, closed eyes, yawning, and no yawning. These categories aim to capture the most common visual signs of drowsiness and fatigue that drivers exhibit, which are crucial for detecting the onset of driver drowsiness. The images were sourced from Kaggle, as well as other publicly available datasets, ensuring a balanced representation of each class. By including various images of drivers in different states of drowsiness, the dataset provides an extensive range of visual cues to train machine learning models.

Each image is carefully labeled to indicate the driver's state, and the dataset includes a mix

of frontal, side, and varied lighting conditions to help the model learn to detect drowsiness under different environmental factors. The images are pre-processed to standardize the size and format, and further augmentation is applied, such as random cropping, rotation, and color jitter, to artificially increase the diversity of the training data. Below are representative samples from the dataset, showcasing the different categories:



Figure 6.2: Open Eyes



Figure 6.3: Closed Eyes

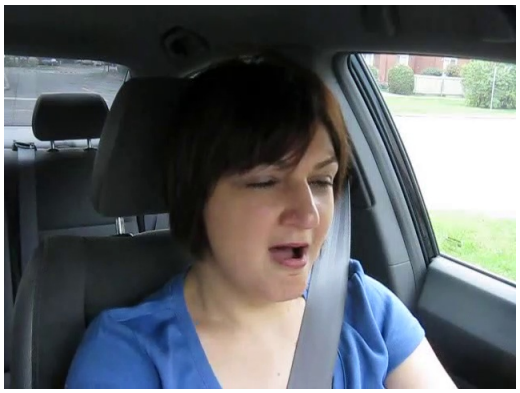


Figure 6.4: Yawning

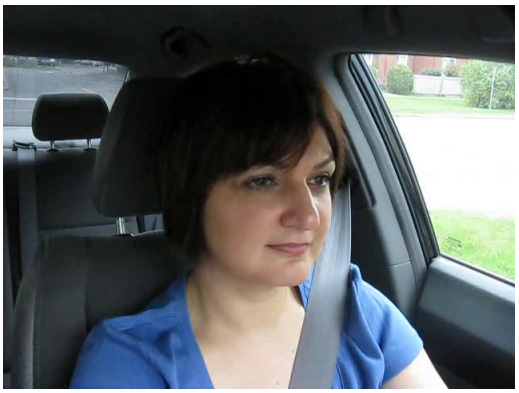


Figure 6.5: No Yawning

Figure 6.6: Representative Samples from the Image Dataset. The dataset includes labeled examples for detecting drowsiness-related states such as open/closed eyes and yawning behaviors.

6.3.2 Sensor Data

The sensor data was collected using mobile device sensors (gyroscope and accelerometer). This data helps in detecting drowsiness through indirect cues such as erratic driving behavior. The dataset includes readings categorized into three classes: aggressive driving, normal driving, and distracted driving. Each class was labeled based on specific driving patterns and was visualized to identify trends and correlations in the data.

Table 6.1: Sample Sensor Data for Driving Behavior Classification

AccX	AccY	AccZ	GyroX	GyroY	GyroZ	Class	Timestamp
-1.6249	-1.0825	-0.2042	-0.0286	0.0513	0.1355	NORMAL	3581630
-0.5947	-0.1224	0.2205	-0.0194	-0.0293	0.0879	NORMAL	3581630
0.7385	-0.2285	0.6677	0.0698	-0.0299	0.0549	NORMAL	3581631
0.1017	0.7776	-0.0667	0.0307	-0.0037	0.0549	NORMAL	3581631
0.5504	-0.7017	0.5148	0.0111	0.0232	-0.0062	AGGRESSIVE	3582826
-0.3632	0.4884	-0.0096	0.0643	0.0385	0.0091	AGGRESSIVE	3582826
-1.1669	-1.3269	-0.1466	0.0380	0.0464	-0.0386	SLOW	3583201
0.1006	0.4301	0.2737	0.0032	0.0489	-0.0416	SLOW	3583202
0.4026	1.0934	-0.5056	0.0594	-0.0220	-0.0477	SLOW	3583202
0.0489	-0.1604	0.2973	-0.0524	0.0715	-0.0001	SLOW	3583203

6.3.3 Future Work: Data Analysis and Model Training

In FYP2, the collected data will be utilized for model training and evaluation. Image data will be processed using Convolutional Neural Networks (CNNs) to detect drowsiness indicators such as closed eyes and yawning. Various architectures, including custom CNNs and MobileNet, will be tested, with the best model selected for mobile deployment. Sensor data from gyroscope and accelerometer readings will be analyzed using ConvLSTM2D models for time-series classification, focusing on driving behaviors like normal, aggressive, and slow driving. Data augmentation will be applied to enhance model performance under diverse conditions, ensuring robustness for real-time deployment.

Chapter 7

Iteration 3

7.1 Model Training and Optimization

7.2 Introduction

The training phase of the *Real-Time Image and Sensors based Driver Drowsiness Detection System* focused on leveraging deep learning techniques to process image and sensor data effectively. This chapter discusses the models explored, their architectures, training processes, performance evaluations, and the challenges faced during this phase.

7.3 Models Used

To achieve accurate predictions, a variety of models were tested and evaluated for image and sensor data.

7.3.1 Image Classification Models

- **Custom CNN:**
  - Computationally expensive and required extensive training.

  - Moderate performance in detecting drowsiness indicators.

- **MobileNet:**
  - Lightweight and highly optimized for mobile devices.
  - Pretrained on ImageNet, capable of detecting complex features with higher accuracy.
  - Achieved superior performance compared to Custom CNN, making it the preferred choice.

7.3.2 Sensor Data Models

To classify driving behavior, the following models were tested:

- Random Forest
- Dense Neural Network (Dense NN)
- Long Short-Term Memory (LSTM)
- CNN-LSTM
- ConvLSTM2D

After evaluation, ConvLSTM2D was selected for its superior accuracy and ability to handle time-series data effectively.

7.4 Model Architectures and Training Details

7.4.1 MobileNet Architecture for Image Classification

- **Base Model:** MobileNetV2 pretrained on ImageNet.
- **Modifications:**

  - Base layers frozen.
  - Added GlobalAveragePooling2D, Dropout (0.5), and a Dense layer with sigmoid activation (1 unit).

- **Optimization:**
  - Optimizer: Adam.
  - Loss Function: Binary Crossentropy.

- **Performance:**
  - Eye State Detection Accuracy: 98%.
  - Yawning Detection Accuracy: 75%.

7.4.2 ConvLSTM2D Architecture for Sensor Data

- **Input:** Time-series data with a shape of (4, 1, 10, 6).

- **Layers:**
  - ConvLSTM2D with 32 filters, kernel size (1, 3), and ReLU activation.
  - Dropout: 0.3.
  - Flatten and Dense layer with 128 units and ReLU activation.
  - Output layer: Dense with 2 units and softmax activation.

- **Optimization:**
  - Optimizer: Adam with a learning rate of 0.001.
  - Loss Function: Binary Crossentropy.

- **Performance:**
  - Accuracy: High accuracy for classifying "Normal" and "Aggressive" driving behaviors.
  - "Slow" driving behavior was dropped during training due to irrelevance in this context.

7.5 Challenges Faced

The model training phase encountered several challenges, including:

- **Latency:** High computational requirements led to delays in real-time predictions.
- **Battery Consumption:** Increased processing power impacted mobile device battery life.
- **Low Light Conditions:** Reduced image quality affected detection accuracy.
- **Dataset Diversity:** Limited variety in training data restricted generalizability.
- **Resource Constraints:** High memory and storage demands for model training.
- **Noisy Data:** Sensor data often contained noise, requiring extensive preprocessing.
- **Imbalanced Classes:** Uneven distribution of data for "Normal" and "Aggressive" classes.
- **Missing Data:** Incomplete sensor readings necessitated handling strategies.

7.6 Conclusion

The model training phase was critical in achieving high-performance metrics for both image and sensor data classification. MobileNet and ConvLSTM2D were chosen as the optimal models for their respective tasks due to their superior accuracy and suitability for real-time implementation. Despite challenges, the trained models demonstrate robust capabilities, paving the way for effective driver drowsiness detection.

Chapter 8

Iteration 4 : FYP 2 Final

8.1 Backend System

The backend system for the *Real-Time Image and Sensors-based Driver Drowsiness Detection System* was designed to process real-time data and run predictions using lightweight TensorFlow Lite models. This system ensures efficient data handling and low latency, enabling seamless integration with the mobile application.

The backend processes two primary inputs:

- **Image Data:** Processed using MobileNet for detecting eye state and yawning.
- **Sensor Data:** Processed using ConvLSTM2D for classifying driving behavior into normal or aggressive categories.

8.2 Code Implementation

Below is the backend implementation in Python, which integrates the TFLite models for image and sensor data processing.

```
from flask import Flask, request, jsonify
import tensorflow as tf
import numpy as np
```

```
4
5  app = Flask(__name__)
6
7  # Load TensorFlow Lite Models
8  interpreter_mobilenet = tf.lite.Interpreter(model_path="
       mobilenet_model.tflite")
9  interpreter_convlstm2d = tf.lite.Interpreter(model_path="
       convlstm2d_model.tflite")
10
11 # Allocate tensors for the interpreters
12 interpreter_mobilenet.allocate_tensors()
13 interpreter_convlstm2d.allocate_tensors()
14
15 # Helper functions for predictions
16 def preprocess_image(image_data):
17     # Resize and normalize image for MobileNet
18     image = tf.image.resize(image_data, [224, 224]) / 255.0
19     return np.expand_dims(image, axis=0).astype(np.float32)
20
21 def preprocess_sensor_data(sensor_data):
22     # Convert sensor data to proper shape for ConvLSTM2D
23     return np.expand_dims(sensor_data, axis=0).astype(np.float32)
24
25 def predict_mobilenet(image_data):
26     input_details = interpreter_mobilenet.get_input_details()
27     output_details = interpreter_mobilenet.get_output_details()
28
29     interpreter_mobilenet.set_tensor(input_details[0]['index'],
       image_data)
30     interpreter_mobilenet.invoke()
31     output = interpreter_mobilenet.get_tensor(output_details[0]['
       index'])
32
33     return "Drowsy" if output[0] > 0.5 else "Awake"
34
35 def predict_convlstm2d(sensor_data):
36     input_details = interpreter_convlstm2d.get_input_details()
```

```
    output_details = interpreter_convlstm2d.get_output_details()

    interpreter_convlstm2d.set_tensor(input_details[0]['index'], sensor_data)
    interpreter_convlstm2d.invoke()
    output = interpreter_convlstm2d.get_tensor(output_details[0]['index'])

    return "Aggressive" if np.argmax(output) == 1 else "Normal"

# Flask API Endpoint
@app.route('/predict', methods=['POST'])
def predict():
    if request.method == 'POST':
        content = request.json
        image_data = np.array(content['image'])  # Expecting raw pixel data
        sensor_data = np.array(content['sensors'])  # Expecting sensor time-series data

        # Preprocess and predict
        processed_image = preprocess_image(image_data)
        processed_sensor = preprocess_sensor_data(sensor_data)

        eye_state = predict_mobilenet(processed_image)
        driving_behavior = predict_convlstm2d(processed_sensor)

        return jsonify({
            "eye_state": eye_state,
            "driving_behavior": driving_behavior
        })

if __name__ == '__main__':
    try:
        app.run(debug=True, host='127.0.0.1', port=5001)
    except SystemExit:
        print("Flask application stopped.")
```

8.3 Backend Workflow

The backend processes data as follows:

1. Receives image and sensor data from the front-end application.
2. Preprocesses the data to meet the input requirements of the respective models.
3. Runs the TensorFlow Lite models to generate predictions for eye state and driving behavior.
4. Combines the predictions to generate a comprehensive report and sends alerts back to the front-end.

8.4 Challenges and Future Improvements

- **Latency:** Optimizing inference time for TFLite models remains a priority to ensure seamless real-time performance.
- **Model Size:** Reducing the size of models while maintaining accuracy for mobile deployment.
- **Scalability:** Ensuring the backend can handle concurrent user requests efficiently.

Future improvements will focus on enhancing backend scalability and exploring additional optimizations for serverless deployment.

Chapter 9

Implementation Details

9.1 Programming Languages and Technologies

9.1.1 Frontend

The mobile application for the drowsiness detection system is developed using the Flutter framework, which employs Dart for frontend development. Flutter enables the creation of a responsive and visually appealing interface, suitable for real-time interaction and user engagement.

9.1.2 Backend

The backend of the application is implemented in Python, leveraging libraries such as TensorFlow Lite, Flask, and NumPy. These technologies ensure efficient model integration, data processing, and API management for real-time predictions.

9.2 System Architecture

The system architecture of the drowsiness detection application follows a client-server model, ensuring seamless interaction between the mobile application and the backend

system. The client-side, built using Flutter, handles the user interface and experience, including features like trip initiation, drowsiness detection alerts, and user-friendly notifications.

Firebase is used for authentication and storage, managing user credentials, trip history, and other essential data. It also provides real-time updates, ensuring a synchronized user experience.

On the server side, the Python-based backend processes sensor and image data, executing real-time inference using pre-trained deep learning models. The models for image-based drowsiness detection (e.g., CNN and MobileNet) and sensor-based behavioral analysis (ConvLSTM2D) are integrated into this backend. The Flask framework facilitates API communication between the mobile app and the backend, ensuring smooth data exchange and command execution.

This modular design ensures maintainability and scalability, as individual components—frontend, backend, and database—can be independently updated and enhanced.

9.3 Database

The application uses Firebase as the primary database, which stores user login details, trip history, and drowsiness detection records. Authentication is handled securely via Firebase, ensuring data privacy and efficient user management.

9.4 Integration of Drowsiness Detection Module

The drowsiness detection system integrates real-time detection modules for both image and sensor data. The image-based detection uses a TensorFlow Lite model trained on datasets from Kaggle, classifying states like closed eyes or yawning. The sensor-based module analyzes gyroscope and accelerometer data using ConvLSTM2D models, identifying driving behaviors such as aggressive or distracted driving. These models are connected to the frontend via APIs developed in Flask, facilitating seamless integration and

real-time performance.

9.5 Error Handling and Validation

The system incorporates robust error handling and validation mechanisms. During login and signup processes, Firebase authentication performs necessary checks to ensure secure access. The application also verifies sensor data and model outputs to prevent incorrect drowsiness alerts. User-friendly error messages are displayed to guide users in resolving issues, enhancing overall reliability.

9.6 Security Considerations

The drowsiness detection system emphasizes security to protect user data. Sensitive information, such as passwords and trip history, is securely handled using Firebase's encryption mechanisms. Communication between the frontend and backend is encrypted, ensuring that data privacy is maintained at all levels of the system.

9.7 User Experience (UX) Considerations

The application prioritizes a user-friendly interface with responsive design, interactive feedback, and real-time alerts. Notifications and visual cues guide the user effectively, ensuring a seamless experience. The app's simplicity and intuitive design make it accessible to a wide audience.

9.8 Version Control

Git and GitHub are employed to track the progress of the application. This version control system enables collaborative development and ensures that code changes are systematically managed and integrated without conflicts.

Chapter 10

Testing and Evaluation

10.1 Overview

The testing and evaluation phase is a critical part of the development process, ensuring that the system meets functional requirements and performs reliably in real-world scenarios. This chapter outlines the methodologies, test cases, and results of the evaluation process for both image-based and sensor-based detection modules. It also includes real-time system testing to verify performance under live conditions. By rigorously testing various components of the system, we aim to validate its robustness, accuracy, and practical applicability in enhancing driver safety.

10.2 API Testing

The drowsiness detection system integrates APIs for real-time communication between the frontend and backend. These APIs handle image-based and sensor-based inputs to classify states such as "Drowsy", "Alert", or driving behaviors like "Aggressive" and "Normal". Testing the API ensures that the system responds accurately and handles unexpected input gracefully.

The following table outlines the test cases designed to evaluate the API's functionality and reliability in various scenarios, such as handling valid and invalid inputs.

Table 10.1: API Test Cases

Test Case	**Objective**	**Method**	**Expected Outcome**
1. Valid Image Data	Ensure correct response for valid image data	POST request to /detect with image containing closed eyes	200 status code, JSON response indicating "Drowsy" or "Alert"
2. Valid Sensor Data	Validate response for sensor data	POST request to /analyze with gyroscope and accelerometer data	200 status code, JSON response with "Aggressive" or "Normal" driving classification
3. Missing Data Field	Test API response for incomplete input	POST request with missing image or sensor data	400 Bad Request status code, message indicating missing required fields
4. Empty Data Input	Assess handling of empty data input	POST request with empty image or sensor data	400 Bad Request status code with error message
5. Large Payload	Verify handling of oversized data input	POST request with a very large image file or sensor dataset	Appropriate response or rejection of payload with error message
6. Special Characters in Request	Evaluate API robustness to malformed requests	POST request with special characters in the JSON payload	400 Bad Request status code
7. Real-Time Inference Timing	Check response time for real-time prediction	POST request with standard data	Response time within predefined threshold (e.g., <1 second)

10.3 Model Performance Testing

The performance of the drowsiness detection system's models was tested using various datasets. Test cases for both image and sensor data were formulated to evaluate the system's accuracy, precision, recall, and F1 score.

Image Data Test Cases

The image-based detection module evaluates visual cues such as closed eyes and yawning to classify driver states. The following Python code outlines test cases for this module:

```
test_cases = [
    {"image_path": "test_images/open_eyes.jpg", "expected_output": "Alert"},
    {"image_path": "test_images/closed_eyes.jpg", "expected_output": "Drowsy"},
    {"image_path": "test_images/yawning.jpg", "expected_output": "Drowsy"},
    {"image_path": "test_images/no_face.jpg", "expected_output": "No Face Detected"}
]
```

Sensor Data Test Cases

The sensor-based detection module uses gyroscope and accelerometer data to classify driving behaviors as "Normal" or "Aggressive". The following table shows the sample sensor dataset used for testing:

Table 10.2: Sample Sensor Data for Driving Behavior Classification

AccX	AccY	AccZ	GyroX	GyroY	GyroZ	Class
-1.6249	-1.0825	-0.2042	-0.0286	0.0513	0.1355	NORMAL
-0.5947	-0.1224	0.2205	-0.0194	-0.0293	0.0879	NORMAL
0.7385	-0.2285	0.6677	0.0698	-0.0299	0.0549	AGGRESSIVE
0.5504	-0.7017	0.5148	0.0111	0.0232	-0.0062	AGGRESSIVE

```
sensor_test_cases = [
    {"sensor_data": [-1.6249, -1.0825, -0.2042, -0.0286, 0.0513,
    0.1355], "expected_output": "Normal"},
    {"sensor_data": [0.5504, -0.7017, 0.5148, 0.0111, 0.0232,
    -0.0062], "expected_output": "Aggressive"},
    {"sensor_data": [-0, 0, -0, 0, 0, -0], "expected_output": "idle
    "}
]
```

Performance Metrics

The performance of the system was evaluated using standard metrics such as accuracy, precision, recall, and F1 score. The following function calculates these metrics:

```
def calculate_metrics(correct_predictions, total_cases,
    false_positives):
    precision = correct_predictions / (correct_predictions +
    false_positives) if correct_predictions + false_positives else 0
    recall = correct_predictions / total_cases if total_cases else
    0
    f1_score = 2 * (precision * recall) / (precision + recall) if
    precision + recall else 0
    return precision, recall, f1_score
```

The following function runs the test cases for sensor-based data:

```
def run_sensor_test_cases(sensor_test_cases):
    correct_predictions = 0
    false_positives = 0
```

```
    for case in sensor_test_cases:
        sensor_data = case["sensor_data"]
        expected_output = case["expected_output"]
        actual_output = analyze_behavior(sensor_data)  # Model's
    prediction
        if actual_output == expected_output:
            correct_predictions += 1
        else:
            false_positives += 1
    total_cases = len(sensor_test_cases)
    accuracy = correct_predictions / total_cases if total_cases
    else 0
    precision, recall, f1_score = calculate_metrics(
    correct_predictions, total_cases, false_positives)
    print(f"Accuracy: {accuracy*100}%")
    print(f"Precision: {precision*100}%")
    print(f"Recall: {recall*100}%")
    print(f"F1 Score: {f1_score*100}%")

run_sensor_test_cases(sensor_test_cases)
```

10.4 Real-Time System Evaluation

To validate the system's real-time performance, the models were deployed on a mobile application and tested under various scenarios. The evaluation focused on:

1. **Response Time:** Ensuring predictions were generated within one second for both image and sensor inputs.
2. **Environmental Conditions:** Testing the image-based module under different lighting conditions and backgrounds.
3. **Driving Behavior:** Validating sensor-based predictions for aggressive, normal, and slow driving across real-world driving patterns.

Chapter 11

User Manual

11.1 Overview

This manual provides a comprehensive guide to using the *Real-Time Image and Sensors Driver Drowsiness Detection System*, a tool designed to enhance driver safety by detecting signs of fatigue and risky driving behavior. Follow these instructions to use the system effectively.

11.2 Getting Started

11.2.1 Account Setup

- **Sign Up:**
  - New users must register by creating an account.
  - Enter your details such as name, email, and password.
- **Log In:**
  - Existing users can log in with their credentials to access the system dashboard.

11.3 Using the System

11.3.1 Real-Time Drowsiness Detection

11.3.2 Image-Based Detection

- Navigate to the **Drowsiness Detection** tab.
- Use the device camera to capture real-time images of the driver.
- Ensure proper lighting and clear visibility of the driver's face for accurate results.
- The system will classify the driver's state as:
  - **Drowsy**: Indicates signs of fatigue such as closed eyes or yawning.
  - **Alert**: No signs of drowsiness detected.

11.3.3 Sensor-Based Detection

- Pair the application with a mobile device or hardware sensors.
- The system collects gyroscope and accelerometer data in real time.
- Driving behavior is categorized as:
  - **Normal Driving**: Stable driving patterns.
  - **Aggressive Driving**: Rapid movements indicating risky behavior.

11.4 Dashboard Features

11.4.1 Accessing and Managing Results

- **Drowsiness Alerts:** Receive real-time notifications on the dashboard.

- **Driving Behavior Reports:** Review classifications of driving patterns in the **Behavior Insights** section.
- **Data History:** Access past alerts and reports from the **Logs** tab for review or analysis.

11.4.2 System Settings

- **Notification Preferences:** Enable or disable audio/visual alerts for drowsiness or risky driving behavior.
- **Threshold Adjustments:** Customize sensitivity settings for image-based and sensor-based detection.

11.5 User Tips and Best Practices

11.5.1 Ensuring Optimal Performance

- Keep the camera lens clean and position it properly to ensure the driver's face is visible.
- Secure sensors to avoid inaccurate readings caused by vibrations.
- Use the system in a stable internet environment to ensure seamless communication between sensors and the application.

11.5.2 Data Privacy

- All collected data is stored securely and is used only for system performance and analysis.
- Users can delete historical data through the **Profile Settings** tab if necessary.

11.6 Troubleshooting

11.6.1 Common Issues

- **Detection Errors:**
  - Ensure the camera view is unobstructed.
  - Check that sensors are calibrated correctly.
- **Connection Problems:**
  - Verify that the device is properly paired with sensors.
  - Restart the application or reconnect the hardware if the issue persists.

11.6.2 Support

For further assistance, visit the **Help** section in the application or contact support via the provided email address.

11.7 Conclusion

This manual ensures that users can effectively use the *Real-Time Image and Sensors Driver Drowsiness Detection System* to enhance driver safety through real-time monitoring and alerts.

Chapter 12

Conclusions and Future Work

12.1 Goals Achieved

12.1.1 Development of a Functional System

A functional system has been developed to detect driver drowsiness using real-time image processing and sensor data. The system combines Convolutional Neural Networks (MobileNet) for detecting facial features like closed eyes and yawning with ConvLSTM2D models for classifying driving behavior based on gyroscope and accelerometer readings. It provides an integrated approach for enhancing driver safety by alerting the driver in case of drowsiness or risky behavior.

12.1.2 Real-Time Monitoring and Notifications

The system successfully demonstrates real-time monitoring capabilities. It processes live data from cameras and sensors to classify states such as drowsy, alert, aggressive driving, or slow driving. Alerts are generated in real-time, ensuring timely feedback to the driver, thus preventing potential accidents.

12.1.3 User-Friendly Interface

A simple and user-friendly interface has been implemented to make the system accessible to a wide range of users. The application provides easy navigation for setting up the system, viewing reports, and customizing alert preferences.

12.2 Future Work

12.2.1 Advanced Model Optimization and Edge Deployment

The current system leverages MobileNet for image-based drowsiness detection, ensuring lightweight and efficient performance on mobile devices. Future optimization efforts will focus on:

- Enhancing MobileNet's inference speed for ultra-low-latency applications on edge devices.
- Further compressing and fine-tuning models without sacrificing accuracy, using techniques like quantization and pruning.
- Exploring advanced neural architecture search (NAS) methods to identify even more efficient model configurations tailored for real-time processing.

12.2.2 Expanded Detection Capabilities

The system currently detects drowsiness and categorizes driving behavior into three classes: normal, aggressive, and slow driving. Future enhancements include:

- Adding detection of additional fatigue indicators, such as head tilts and blinking patterns.
- Improving classification granularity for driving behavior to include events like sudden braking and sharp turns.

12.2.3 Integration with Vehicle Systems

Future iterations of the system will focus on integrating with vehicle hardware, such as onboard cameras and sensor modules. This integration would enable:

- Automatic vehicle interventions, such as slowing down or sounding alarms when drowsiness is detected.
- Recording of driving behavior data for compliance or training purposes.

12.2.4 Improved Data Collection and Augmentation

To improve the accuracy and robustness of the system, future efforts will involve:

- Collecting a larger and more diverse dataset that includes various driving conditions, lighting, and driver demographics.
- Applying advanced data augmentation techniques to simulate real-world challenges, such as low-light scenarios or obstructions in the camera view.

12.2.5 Field Testing and Deployment

The system will be subjected to extensive testing in real-world conditions. Feedback from users will be incorporated to refine the system's performance and usability. After successful field trials, the system will be deployed for commercial and consumer use.

12.2.6 Collaboration with Industry Partners

To maximize the impact of the system, collaborations with automotive manufacturers and fleet management companies will be pursued. These partnerships can facilitate large-scale deployment and integration into modern vehicles.

12.3 Conclusion

The *Real-Time Image and Sensors-based Driver Drowsiness Detection System using Deep Learning* addresses a critical challenge in road safety by effectively detecting driver fatigue and risky driving behaviors. By combining real-time image analysis with sensor data processing, the system demonstrates the potential to reduce accidents caused by drowsiness and inattentive driving.

The use of lightweight models, such as MobileNet, ensures the system's compatibility with mobile and embedded devices, making it accessible and efficient for real-world applications. Its ability to detect visual signs of fatigue and classify driving behavior as normal or aggressive adds to its versatility and practical utility.

The project represents a significant advancement in utilizing deep learning for real-time driver monitoring and sets a foundation for future innovations in intelligent transportation systems.

Bibliography

[1] F. Asim and S. Sunil, "DrowsyDetect: Android Application for Driver Drowsiness Detection Using Eye Closure Rate with Deep Learning," *Journal of Student Research*, 2023. Available: https://www.jsr.org/index.php/path/article/view/2152.

[2] F. Ridwan and L. P. Hung, "Driver-Drowsiness Detection System Using Deep Learning (CNN)," in *Proc. 2023 IEEE Int. Conf. on Artificial Intelligence in Engineering and Technology (IICAIET)*, Kota Kinabalu, Malaysia, 2023, pp. 78–83, doi: 10.1109/IICAIET59451.2023.10291637.

[3] M. Shereesha, G. Sreehitha, G. R. Reddy, J. Srikanth, and H. R. Varma, "Driver Drowsiness Detection using Convolutional Neural Networks (CNNs)," in *Proc. 2023 4th IEEE Global Conf. for Advancement in Technology (GCAT)*, Bangalore, India, 2023, pp. 1–5, doi: 10.1109/GCAT59970.2023.10353337.

[4] S. S. Bharti, R. Mahajan, S. Bhoriya, and A. Bhat, "Driver's Drowsiness Detection using MobileNet on Web," in *Proc. 2023 7th Int. Conf. on Intelligent Computing and Control Systems (ICICCS)*, Madurai, India, 2023, pp. 1229–1233, doi: 10.1109/ICICCS56967.2023.10142379.

[5] A. Ritesh, N. Jagatia, and P. Deshmukh, "Driver Drowsiness Detection Using Machine Learning," in *Proc. 2023 6th Int. Conf. on Advances in Science and Technology (ICAST)*, Mumbai, India, 2023, pp. 100–105, doi: 10.1109/ICAST59062.2023.10455018.

[6] R. Jabbar, M. Shinoy, M. Kharbeche, K. Al-Khalifa, M. Krichen, and K. Barkaoui, "Driver Drowsiness Detection Model Using Convolutional Neural Networks Tech-

niques for Android Application," in *Proc. 2020 IEEE Int. Conf. on Informatics, IoT, and Enabling Technologies (ICIoT)*, Doha, Qatar, 2020, pp. 237–242, doi: 10.1109/ICIoT48696.2020.9089484.

[7] T. M. Khan, H. Anwar, F. Ullah, A. U. Rehman, R. Ullah, A. Iqbal, B.-H. Lee, and K. S. Kwak, "Smart Real-Time Video Surveillance Platform for Drowsiness Detection Based on Eyelid Closure," *Wireless Communications and Mobile Computing*, vol. 2019, p. 2036818, 2019, doi: 10.1155/2019/2036818.

[8] I. Cojocaru and P.-S. Popescu, "Building a Driving Behaviour Dataset," *University of Craiova*, 2023, 13 A.I.Cuza Street, 200585 Craiova, Romania.

[9] I. Cojocaru, P.-S. Popescu, and C. Mihaescu, "Driver Behaviour Analysis based on Deep Learning Algorithms," *University of Craiova*, 2023, 13 A.I.Cuza Street, 200585 Craiova, Romania.